Aula 01 - Fundamentos de Manutenção de Software

Disciplina: Manutenção de Software

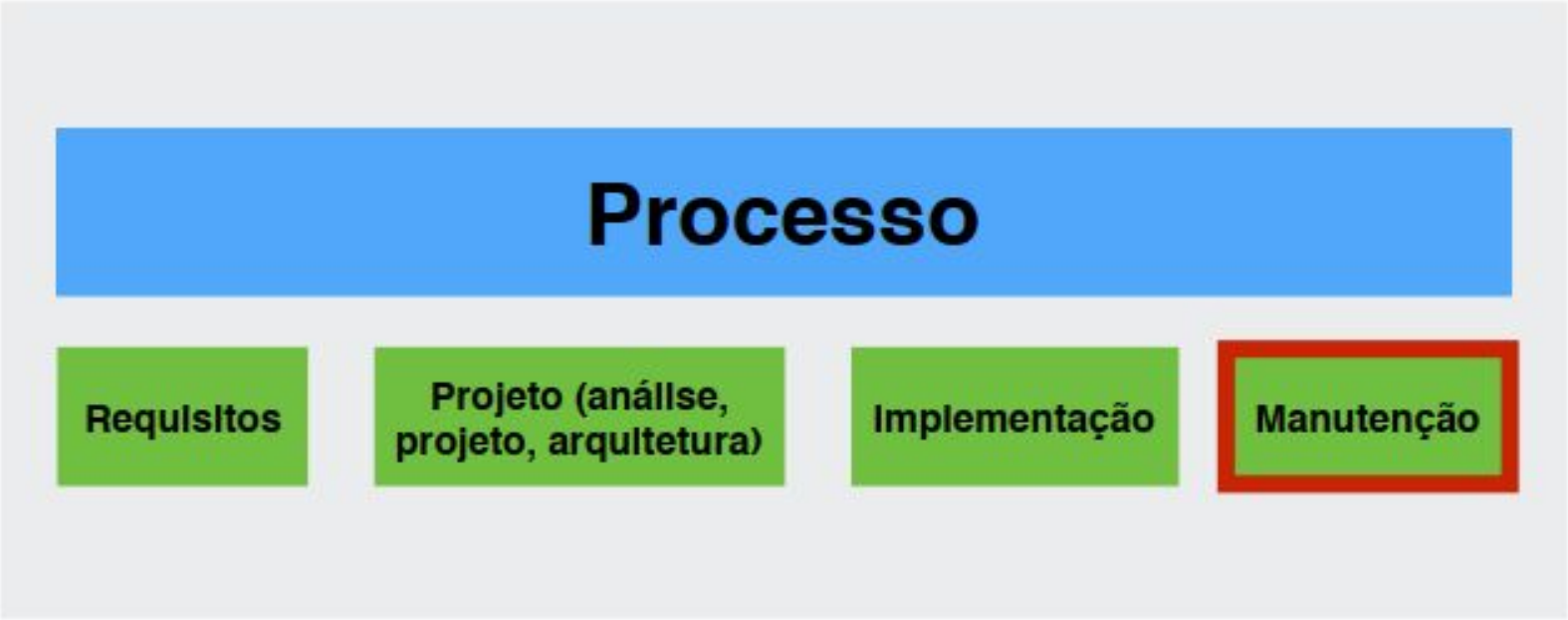
Prof. Me. João Paulo Biazotto

UniCesumar

Agenda

- Revisão de processos de software
- Tipos de manutenção
- Leis de Lehman

Processo de Software



Processo de Software

- **Especificação:** Defina a funcionalidade do software e as restrições sobre sua operação.
- **Projeto e implementação:** O software deve ser desenvolvido conforme a especificação.
- **Validação:** O software deve ser validado para assegurar que atenda às expectativas do cliente.
- **Evolução:** O software deve evoluir para atender aos novos requisitos que surgirem.

Manutenção/Evolução

- Disciplina responsável por lidar com as **mudanças relacionadas** ao software depois da sua **entrega.**
- Software: código + documentação (requisitos, análise, projeto, manuais, etc)
- "Ato de manter uma entidade em bom estado de reparo, eficiência ou validade, para evitar falhas ou declínio" (IEEE)

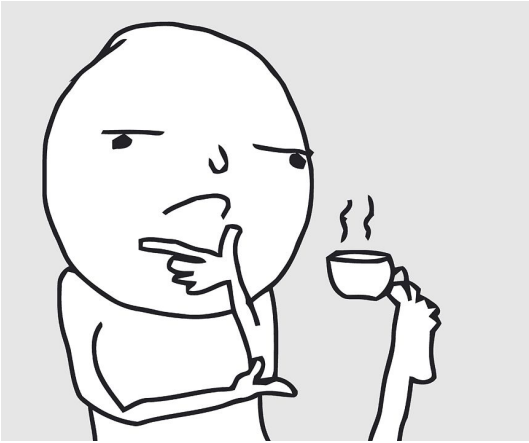
Desenvolvimento x Manutenção

- Adicionar uma nova funcionalidade durante o desenvolvimento é mais fácil que durante a manutenção

- Manutenção de software deve respeitar certos parâmetros e restrições existentes

Funcionalidades

Por que é mais fácil adicionar uma funcionalidade durante o desenvolvimento?



Desenvolvimento x Manutenção

- Quando realiza-se mudança em uma casa
  - Arquitetos e construtores devem ter cuidado para não enfraquecer a estrutura da casa.
  - Custo de um novo cômodo pode ser barato, mas o custo por metro quadrado é muito mais alto.
  - Isso ocorre pois é necessário a remoção de paredes, alteração do encanamento/fiação, etc.

Desenvolvimento x Manutenção

- Em software, ao projetar uma nova funcionalidade, o mantenedor deve investigar o sistema atual para abstrair sua arquitetura e detalhes de baixo nível:
  - Realizar como a mudança será acomodada
  - Prever o impacto da mudança (efeito cascata)
  - Determinar as características necessárias para o trabalho

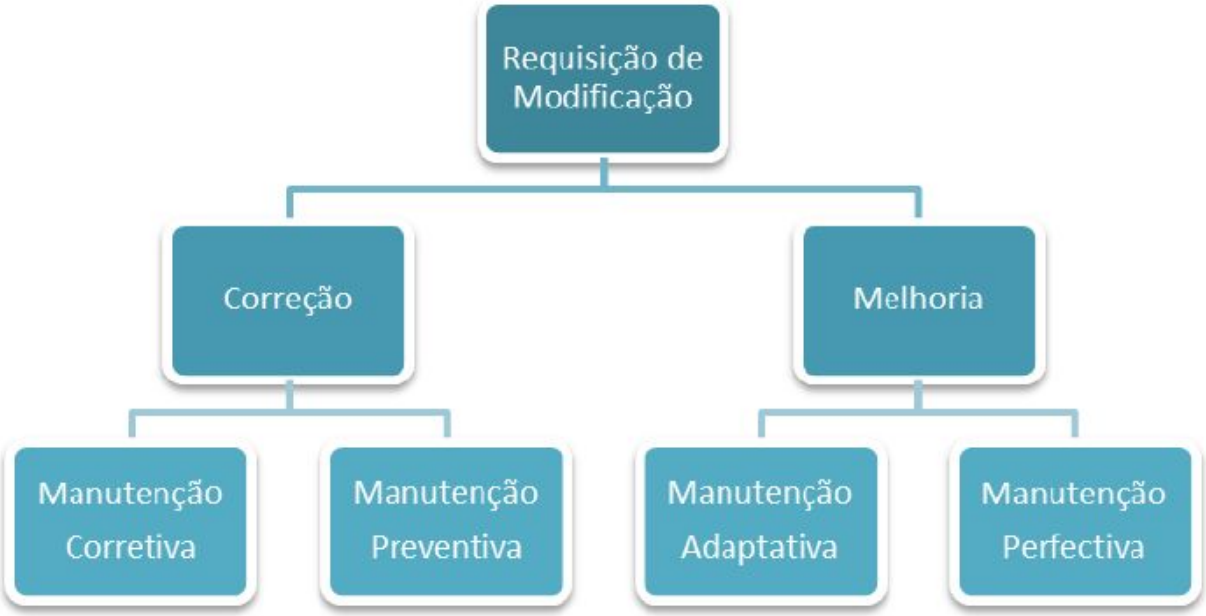
Quando realizar manutenção de software?

- Adicionar novas funcionalidades
- Corrigir defeitos
- Melhorar design
- Comunicar com outros sistemas
- Migrar de SO, BD, bibliotecas, etc
- Adaptar a diferentes hardwares
- Adaptar a leis, regras de negócio, etc
- Refatorar o código

Categorias de Manutenção

- Manutenção Corretiva
- Manutenção Preventiva
- Manutenção Adaptativa
- Manutenção perfectiva ou Evolutiva

Categorias de Manutenção



Manutenção Corretiva

- Modificações no software para corrigir defeitos
- Ex. defeitos em requisitos, projeto, código
- Pode gerar outros problemas como aumento de complexidade e outros efeitos cascatas

Manutenção Preventiva

- Modificações no software para prevenir potenciais problemas no futuro
- Lida com o deterioramento de estruturas, previne falhas e melhorar a manutenibilidade
- Torna os programas mais fáceis de entender e facilita trabalhos de manutenção futuros
- Ex. reestruturação de código, otimização de código, refatoração, atualização de documentação

Manutenção Adaptativa

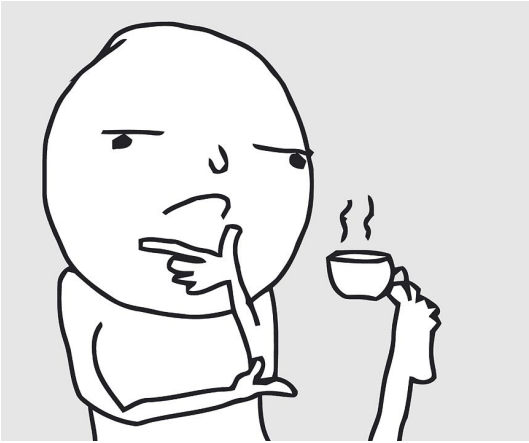
- Adaptações para manter o software usável devido às alterações no ambiente externo
- Ocorre pois o ambiente está em constante evolução, mesmo quando defeitos não existem
- Ex. alterações no SO, BD, servidor, compilador, bibliotecas, frameworks, hardware
- Se SO é atualizado, mudanças podem ser realizadas no software para acomodar o novo SO

Manutenção Evolutiva

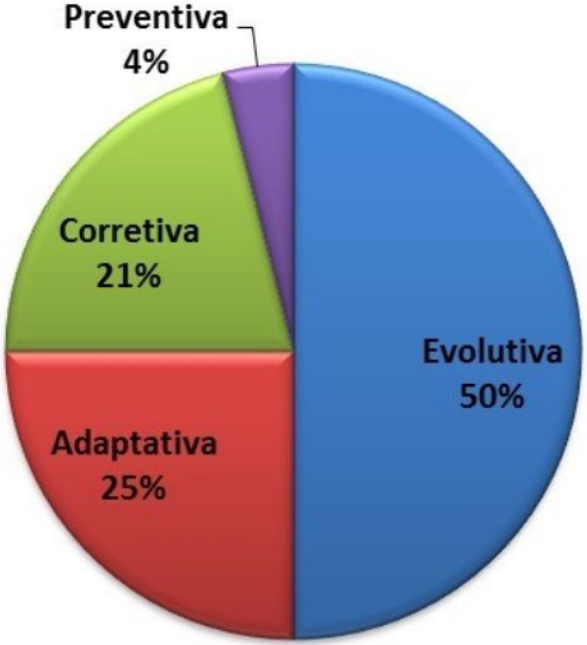
- Modificações para fornecer melhorias aos usuários
- Expande os requisitos do sistema
- Quando o software se torna útil, os usuários solicitam melhorias além do escopo inicial
- Ex. novas funcionalidades

Categorias de Manutenção

Quanto de cada tipo de manutenção é mais feita?



Categorias de Manutenção



Categorias de Manutenção

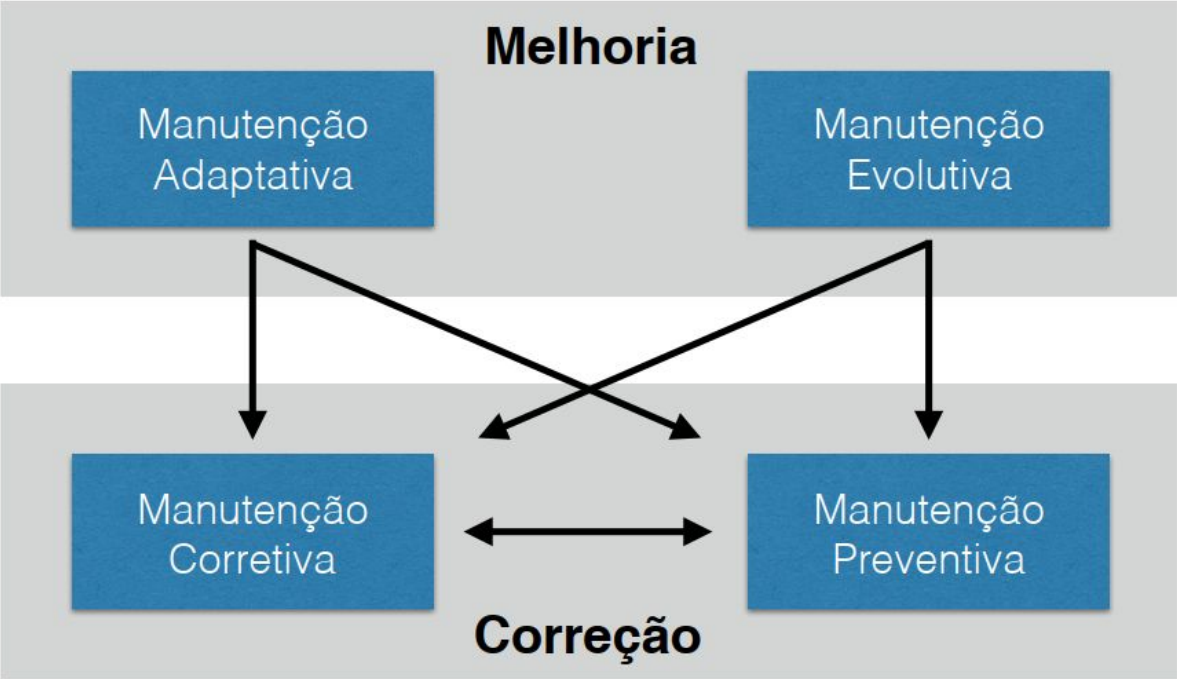
- Manutenção **corretiva** e **evolutiva** são mais visíveis e trazem valor direto para o usuário

- Manutenção **preventiva** e **adaptativa** trazem valor indireto para o usuário (por quê?)

Relação entre os tipos de manutenção

- Manutenções são categorizadas de forma individual, mas na prática estão interligadas
- Ao modificar o código devido a uma nova biblioteca (**adaptativa**), defeitos podem ser introduzidos. Logo, esses defeitos devem ser corrigidos (**corretiva**)
- A introdução de uma nova funcionalidade (**evolutiva**), pode requerer que o código seja refatorado antes para facilitar sua implementação (**preventiva**)

Relação entre os tipos de manutenção



Leis de Lehman

- O termo manutenção de software data dos anos 60/70
- Lehman após 20 anos de pesquisas, formulou as Leis da evolução (8 leis da evolução)

Leis de Lehman

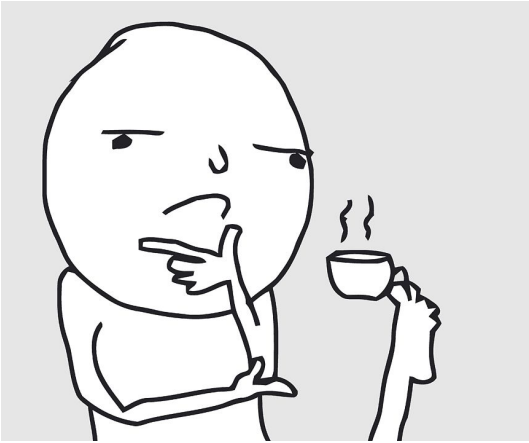
- **Mudança contínua:** software deve ser continuamente adaptado, senão torna-se menos satisfatório
- **Complexidade crescente:** se não forem tomadas medidas para reduzir complexidade, ela irá aumentar progressivamente
- **Crescimento contínuo:** software deve ter funcionalidades ampliadas para manter a satisfação dos seus usuários; o projeto inicial não inclui tudo e deve ser aumentado
- **Declínio de qualidade:** software perde qualidade se não é alterado para refletir as mudanças do ambiente externo

Leis de Lehman

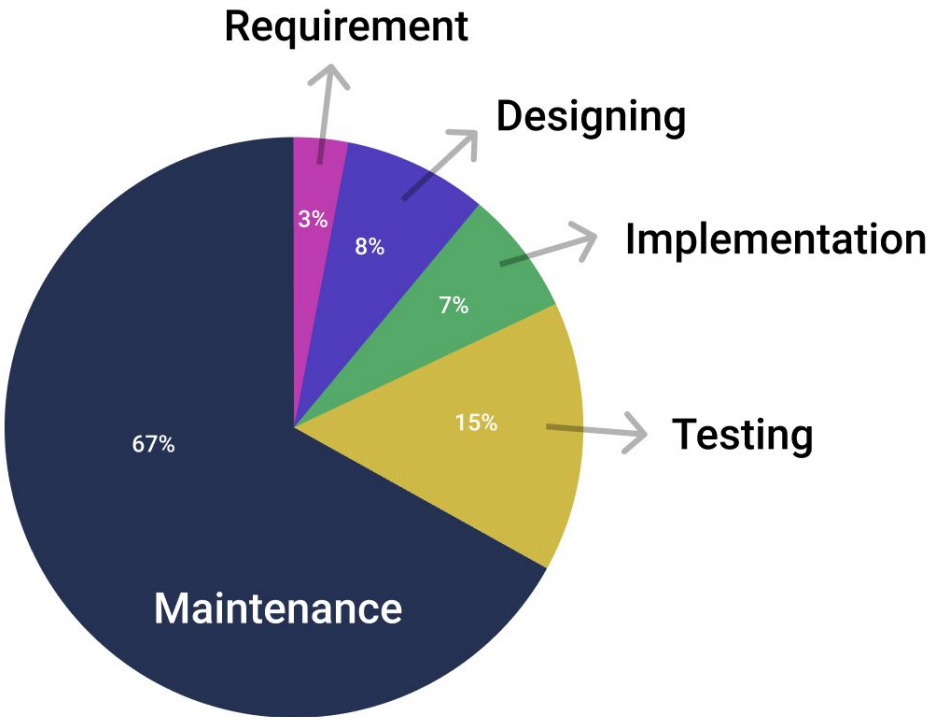
- **Auto regulação:** software tende a se auto regular, seguindo padrões previsíveis e comportamentos disciplinados.
- **Conservação da estabilidade organizacional:** O software tende a se manter em um nível constante em relação à organização
- **Conservação da familiaridade:** Muitas mudanças no software afetam a familiaridade dos usuários com o software
- **Sistema de feedback:** Cada nova versão, gera feedback e novas informações e ajustes

Custo de Manutenção de Software

Qual o custo de manutenção do software?

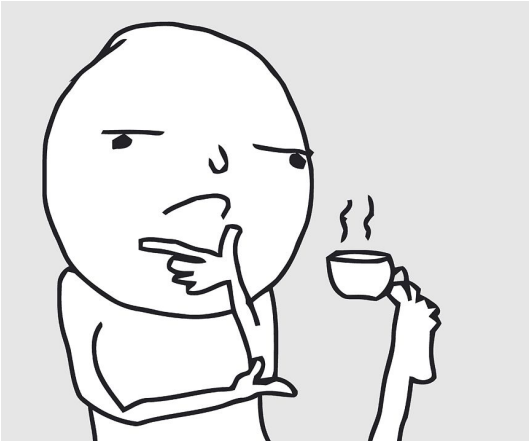


Custo de Manutenção de Software



Custo de Manutenção de Software

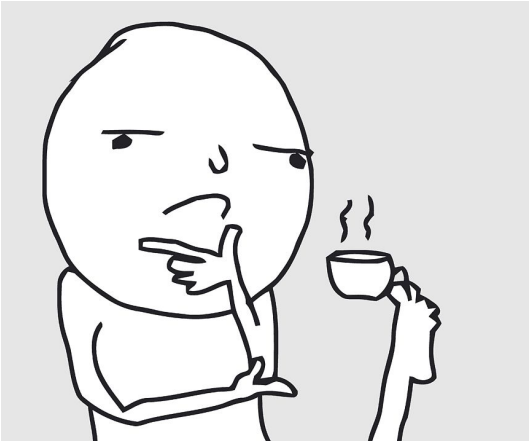
Por que manutenção de software é tão caro?



Custo de Manutenção de Software

Por que manutenção de software é tão caro?

BAIXA MANUTENIBILIDADE!

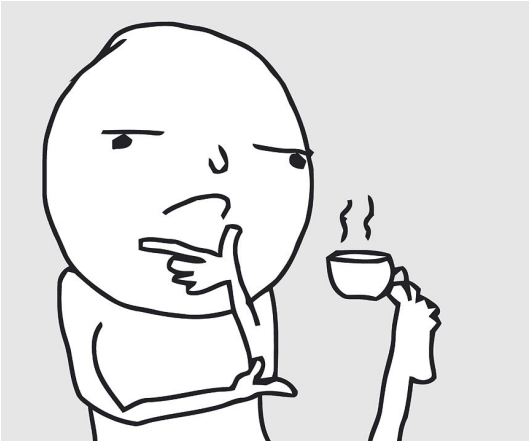


Manutenibilidade

- Facilidade de manutenção de um sistema
- Difícil de quantificar
- Existem métricas e aspectos do sistema podem ser medidos (ex: complexidade, portabilidade)
- Entretanto não existe fórmula mágica.

Manutenibilidade

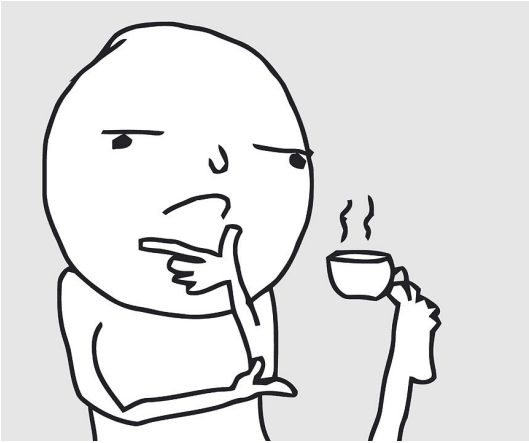
O que impacta a manutenibilidade de um sistema?



Manutenibilidade

O que impacta a manutenibilidade de um sistema?

- **Qualidade de código**
- **Arquitetura**
- **Acoplamento**
- **Etc...**



Métricas de Manutenibilidade

- Métricas de Chidamber-Kemerer (CK): específicas para OO
  - Profundidade de Herança (DIT)
  - Número de filhos (NOC)
  - Acoplamento entre objetos (CBO)
  - Falta de coesão em métodos (LCOM)
  - Métodos ponderados por classe (WMC)
  - Resposta para classe (RFC)

DÚVIDAS?

Aula 02 - Domain-Driven Design e Linguagem Onipresente

Disciplina: Manutenção de Software

Prof. Me. João Paulo Biazotto

UniCesumar

Agenda

- Revisão dos tipos de manutenção
- Domain-driven design
- Linguagem Onipresente
- Atividade

Categorias de Manutenção

- Manutenção Corretiva
- Manutenção Preventiva
- Manutenção Adaptativa
- Manutenção perfectiva ou Evolutiva

Domain-driven design (DDD)

- Abordagem para desenvolvimento de software que visa alinhar o código com o domínio do negócio, priorizando a compreensão e a resolução dos problemas desse domínio.

Domain-driven design (DDD)

- Propõe uma série de conceitos e práticas para melhorar a modelagem e a implementação de um sistema, levando em consideração a complexidade do negócio e a colaboração entre desenvolvedores e especialistas do domínio.

Domain-driven design (DDD)

1. Compreender o domínio do negócio
2. Identificar os conceitos fundamentais (modelagem do domínio)
3. Modelagem do domínio
4. Definição de bounded contexts (contextos delimitados)
5. Design da arquitetura
6. Implementação do código

Compreender o domínio do negócio

- A primeira etapa é adquirir um conhecimento aprofundado do **domínio do negócio**.
- Isso envolve trabalhar em estreita **colaboração** com especialistas do domínio, como usuários, analistas de negócio ou **stakeholders**.
- O objetivo é entender as **regras de negócio**, os processos, as interações e as terminologias utilizadas no domínio em questão.

Identificar os conceitos fundamentais

- Com base na compreensão do domínio do negócio, identifique os **conceitos e as entidades fundamentais** que são essenciais para o sistema.
- Esses conceitos podem ser representados por **classes ou objetos** que encapsulam o comportamento e as informações relacionadas a eles.
- Essa modelagem do domínio é um aspecto crítico do DDD, pois ajuda a estabelecer uma **linguagem comum** entre especialistas do domínio e desenvolvedores.

Modelagem do domínio

- Com base no conhecimento adquirido e na identificação do **domínio**, você **modela o domínio do problema** em termos de entidades, agregados, objetos de valor e regras de negócio.
- Essa modelagem deve **refletir a linguagem do domínio**, ou seja, utilizar termos e conceitos do negócio em vez de termos técnicos. Diagramas, como Diagramas de Domínio ou **Diagramas de Classes**, podem ser utilizados para visualizar a estrutura e as relações entre os elementos do domínio.

Definição de contextos delimitados

- Um **bounded context** (contexto delimitado) é uma fronteira lógica que define um contexto claro e limitado em que um **conjunto específico de entidades e regras** de negócio operam.
- Nesta etapa, você identifica e delimita os bounded contexts no domínio, levando em consideração os limites **de responsabilidade e as interações** entre as partes do sistema.

Design da arquitetura

- Projetar a arquitetura do sistema, identificando as diferentes camadas (como a camada de **aplicação**, a camada de **domínio**) e as formas de comunicação e integração entre elas.
- É importante **considerar os padrões e princípios arquiteturais**, como o princípio de inversão de dependência (Dependency Inversion Principle) e a separação de preocupações (**Separation of Concerns**).

Implementação do código

- Com a arquitetura definida, você pode começar a implementar o código do sistema, seguindo as práticas e padrões de codificação apropriados.
- É fundamental manter o alinhamento com a modelagem do domínio e utilizar a linguagem do domínio na implementação, tornando o código mais expressivo e próximo do problema real.

Linguagem Onipresente

- A "Linguagem Onipresente" (Ubiquitous Language, em inglês) é um conceito introduzido por Eric Evans no livro "Domain-Driven Design" (Projeto Orientado a Domínio).
- A ideia por trás da Linguagem Onipresente é que desenvolvedores e especialistas de um determinado domínio compartilhem uma linguagem comum para descrever e discutir o sistema que estão construindo.

Linguagem Onipresente

- A Linguagem Onipresente é uma linguagem rica em termos de negócio, expressões e conceitos que refletem o domínio em que o software está sendo desenvolvido.
- Ela deve ser compreensível tanto pelos especialistas do domínio quanto pelos desenvolvedores, criando uma ponte de comunicação efetiva entre eles.

Benefícios da Linguagem Onipresente

- **Comunicação eficaz:** Todos os envolvidos no projeto, incluindo especialistas do domínio e desenvolvedores, falam a mesma linguagem, o que facilita a comunicação, a compreensão mútua e evita mal-entendidos.
- **Modelagem do domínio:** A Linguagem Onipresente ajuda a capturar e expressar o conhecimento do domínio no código-fonte. Isso permite que o modelo de domínio seja refletido de forma mais precisa e semântica no software.

Benefícios da Linguagem Onipresente

- **Maior produtividade:** A linguagem compartilhada e compreendida por todos facilita o desenvolvimento, a manutenção e a evolução do software. Isso reduz o tempo gasto em traduções entre diferentes terminologias e melhora a colaboração em equipe.
- **Evolução contínua:** A Linguagem Onipresente evolui junto com o domínio do negócio. À medida que os especialistas do domínio e os desenvolvedores trabalham em conjunto, a linguagem é enriquecida e refinada, permitindo uma modelagem de software cada vez mais precisa e alinhada com as necessidades do negócio.

Benefícios da Linguagem Onipresente

- É importante destacar que a Linguagem Onipresente não é apenas sobre a escolha de palavras, mas também sobre o entendimento profundo do domínio e a criação de um modelo de domínio compartilhado e bem definido.

- Ela é uma parte essencial do Domain-Driven Design (DDD) e pode contribuir significativamente para o sucesso de um projeto de software.

DÚVIDAS?

Aula 03 - Princípios SOLID

Disciplina: Manutenção de Software

Prof. Me. João Paulo Biazotto

UniCesumar

Agenda

- Revisão de DDD
- Princípios SOLID
- Princípio DRY
- Lei de Demeter

Domain-driven design (DDD)

- Abordagem para desenvolvimento de software que visa alinhar o código com o domínio do negócio, priorizando a compreensão e a resolução dos problemas desse domínio.

Domain-driven design (DDD)

1. Compreender o domínio do negócio
2. Identificar os conceitos fundamentais (modelagem do domínio)
3. Modelagem do domínio
4. Definição de bounded contexts (contextos delimitados)
5. Design da arquitetura
6. Implementação do código

Por que falar de SOLID?

- Sistemas crescem: mais regras, mais integrações, mais "gambiarras"
- O problema não é mudar o software — é mudar sem quebrar tudo
- SOLID = conjunto de princípios para:
  - reduzir acoplamento
  - aumentar coesão
  - facilitar evolução e testes
- Objetivo da aula: **entender o que é, por que existe e quando aplicar**

Por que falar de SOLID?

- Temos um fluxo simples: calcular total + aplicar desconto + cobrar + enviar e-mail
- No início funciona... até começarem as mudanças:
  - novos tipos de desconto (cupom, fidelidade, Black Friday)
  - novas formas de pagamento (Pix, cartão, boleto)
  - novas notificações (e-mail, SMS, push)
  - logs/auditoria obrigatória
- Pergunta
  - Como evoluir sem transformar tudo em um “monstro”?

Por que falar de SOLID?

- CheckoutService.finalizarPedido(pedido) faz tudo:
  - valida estoque
  - calcula total
  - aplica desconto
  - processa pagamento
  - envia e-mail
  - registra log/auditoria

Qual o problema?

- classe gigante, difícil de entender
- testes difíceis (precisa de gateway de pagamento + SMTP)
- mudanças em “desconto” exigem mexer no checkout inteiro

Requisitos novos viram cascata de if/else

- Mudança 1: adicionar Pix e “comprovante”
- Mudança 2: desconto por categoria + desconto progressivo
- Mudança 3: notificar por SMS quando for Pix

Requisitos novos viram cascata de if/else

- if (pagamento == CARTAO) ... else if (pagamento == PIX) ...
- Cada mudança aumenta:
  - complexidade
  - risco de bug
  - retrabalho de testes

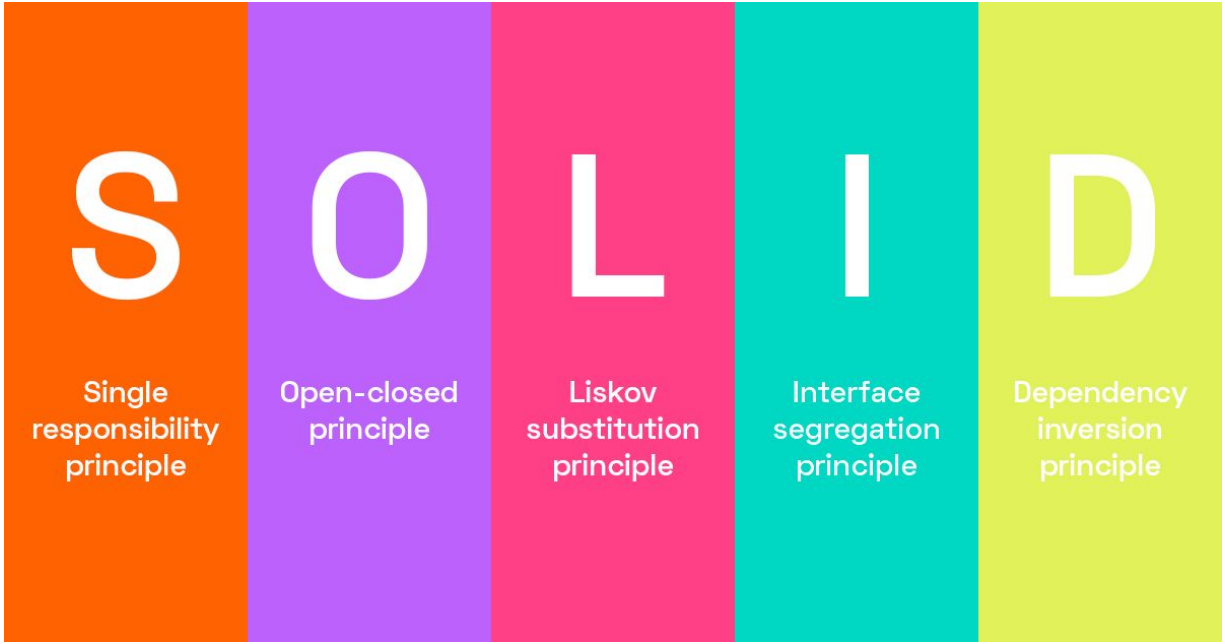
O que está por trás da bagunça?

- Causas principais
  - muitas responsabilidades no mesmo lugar (uma classe faz “tudo”)
  - dependência direta de detalhes (gateway, email, sms dentro do checkout)
  - extensão por edição (toda nova regra obriga alterar código existente)
  - interfaces inchadas (serviço obrigado a conhecer métodos que nem usa)
- No fim das contas
  - O problema não é OO, mas sim o design sem princípios para mudança constante.

Como solucionar o problema?

- Separar “o que muda junto” em componentes:
  - CalculadoraDeTotal
  - PoliticaDeDesconto (várias implementações)
  - ProcessadorDePagamento (Pix, Cartão, Boleto)
  - Notificador (Email, SMS, Push)
  - Fazer o Checkout depender de abstrações, não de detalhes

O que é SOLID?


S
Single responsibility principle
O
Open-closed principle
L
Liskov substitution principle
I
Interface segregation principle
D
Dependency inversion principle

Ideia central de cada princípio

- SRP: uma classe/módulo tem um motivo principal para mudar
- OCP: evoluir por extensão, não por alteração constante do código existente
- LSP: subtipos devem ser substituíveis sem surpresas (contratos respeitados)
- ISP: interfaces pequenas e focadas; evitar “interfaces gordas”
- DIP: depender de abstrações, não de implementações (inversão de dependência)

DÚVIDAS?

Aula 04 - Program Comprehension e Métricas de Complexidade

Disciplina: Manutenção de Software

Prof. Me. João Paulo Biazotto

UniCesumar

Compreensão de programas

Atividades para entender o software existente

Mudanças:

- Corretiva
- Preventiva
- Evolutiva
- Adaptativa

Compreensão de programas

Antes de implementar qualquer mudança é necessário:

- Conhecer o software como um todo
- Conhecer os detalhes relacionados à mudança

Compreensão de programas - Etapas

- Conhecimento geral sobre o que o software faz
- Identificar onde as alterações devem ser realizadas
- Conhecimento mais profundo sobre como as partes a serem alteradas funcionam

Compreensão de programas - Etapas

- Compreensão de programas consome uma proporção significativa de esforço de manutenção

- Dados podem variar de acordo com a qualidade do código, documentação, nível de experiência do desenvolvedor, tipo da correção, etc

Compreensão de programas - Etapas

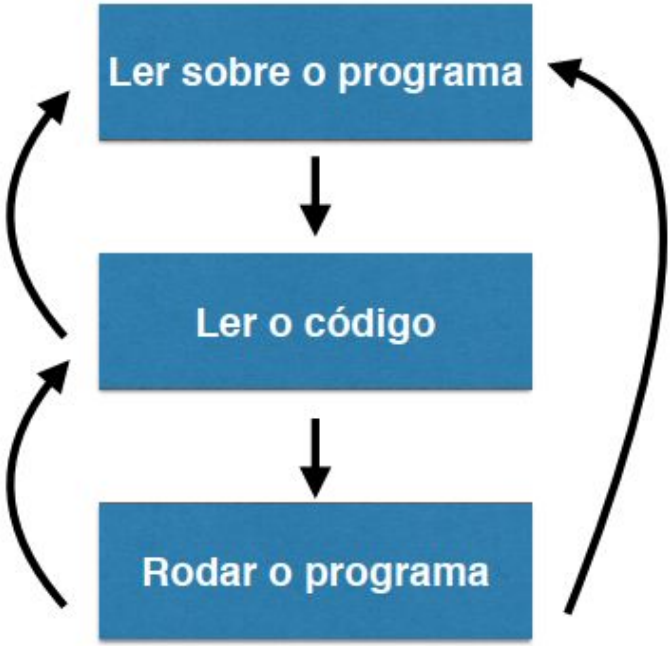
- Isso envolve lidar com certos aspectos:
- Domínio do problema (para estimar esforço)
  - Efeito de execução (verificar se mudança teve efeito desejado)
  - Relação causa-efeito (análise de impacto)
  - Ferramentas de suporte (métricas de complexidade, manutenabilidade, engenharia reversa)

Modelo de Compreensão

De modo geral são três tarefas:

- **Ler sobre o programa**: acessar documentação (especificação, projeto) se disponível
- **Ler o código:** obter uma visão global e local do programa, usar ferramentas de engenharia reversa
- **Rodar o programa:** revelar características do sistema difíceis de obter apenas lendo o código

Modelo de Compreensão



Fatores que podem afetar a compreensão de programas

- O que torna um programa mais difícil de entender?

Fatores que podem afetar a compreensão de programas

- Formatação do código

```
<?php
class Calculator {
public function add($a, $b) { return $a + $b; }
public function multiply($a, $b) { return $a * $b;
} public function divide($a, $b) { if($b == null) {
throw new Exception("Division by zero"); } return $a / $b; }
public function subtract($a, $b) { return $a - $b; } }
?>
```

Fatores que podem afetar a compreensão de programas

- Formatação do código

```php
<?php
class Calculator {
    public function add($a, $b) {
        return $a + $b;
    }
    public function multiply($a, $b) {
        return $a * $b;
    }
    public function divide($a, $b) {
        if ($b == null) {
            throw new Exception ( "Division by zero" );
        }
        return $a / $b;
    }
    public function subtract($a, $b) {
        return $a - $b;
    }
}
?>
```

Fatores que podem afetar a compreensão de programas

- Conhecimento sobre o domínio
- Suporte de ferramentas de engenharia reversa
- Qualidade do código (legibilidade, complexidade, uso de padrões de projeto, etc)
- Qualidade da documentação (interna e externa)
- Nível de experiência do desenvolvedor

Fatores que podem afetar a compreensão de programas

- **Conhecimento sobre o domínio**
  - Ao longo do tempo, desenvolvedores tornam-se especialistas em um determinado domínio (LP, BD, bibliotecas, etc)

Fatores que podem afetar a compreensão de programas

- **Conhecimento sobre o domínio**
  - Logo, o nível de expertise tem um papel significante na compreensão de programas
  - Estudos mostram que o nível de expertise influencia positivamente no entendimento

Fatores que podem afetar a compreensão de programas

- **Suporte de ferramentas de engenharia reversa**
- Exemplos:
  - Moose, Jarchitect, SonarQube, MaintainJ, Findbugs, Lint, Newrelic, IDA Pro, Hiew, Scylla

Fatores que podem afetar a compreensão de programas

- **Qualidade do código (legibilidade, complexidade, uso de padrões de projeto, etc)**
  - Complexidade
  - Legibilidade
  - Comentários em código
  - Formatação

Qualidade do código - Complexidade

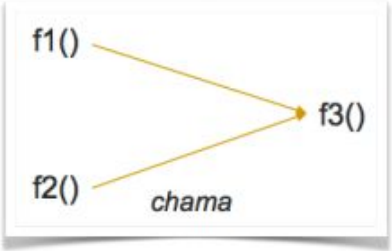
- Exemplos de métricas:
  - Fan-in
  - Fan-out
  - Complexidade ciclomática
  - Tamanho do código
  - Profundidade de aninhamento
  - Tamanho do vocabulário

Qualidade do código - Complexidade

- Exemplos de métricas:
  - **Fan-in**
  - **Fan-out**
  - **Complexidade ciclomática**
  - Tamanho do código
  - Profundidade de aninhamento
  - Tamanho do vocabulário

Fan-in

- Número de funções que chamam uma dada função
- Valor alto significa grande impacto em mudanças (propagação)



fan-in(f3) = 2

Fan-out

- Número de funções chamadas pela função
- Valor alto significa grande complexidade da função

Fan-in e Fan-out

- Exemplo
- Suponha que temos os seguintes módulos:
- - Módulo A: chamado pelos módulos B, C e D.
- - Módulo B: chama os módulos E e F.
- - Módulo C: chama os módulos G e H.
- **Calcule Fan-in de A e Fan-out de B e C**

Fan-in e Fan-out

- Calcule Fan-in de A e Fan-out de B e C
  - Fan-in de A: 3 (B, C, D)
  - Fan-out de B: 2 (E, F)
  - Fan-out de C: 2 (G, H)

Fan-in e Fan-out

- Dado o seguinte sistema:
  - Módulo X: chamado por A, B.
  - Módulo Y: chamado por X.
  - Módulo Z: chamado por X, Y.
  - Módulo W: chama X, Y, Z.

- - Calcule o **Fan-in** e o **Fan-out** para os módulos X, Y, Z, e W.

Fan-in e Fan-out

- Fan-in de X: 3 (A, B, W)
- Fan-out de X: 2 (Y, Z)
- Fan-in de Y: 2 (X, W)
- Fan-out de Y: 1 (Z)
- Fan-in de Z: 3 (X, Y, W)
- Fan-out de Z: 0
- Fan-in de W: 0
- Fan-out de W: 3 (X, Y, Z)

Complexidade ciclomática

- Indica a complexidade do código através da quantidade de caminhos de execução
- Mede a complexidade de controle do programa
  - if, while, for, etc
- Relacionada à facilidade de compreensão

Complexidade ciclomática

- Mede a complexidade de um programa com base no número de caminhos lineares independentes pelo código.
- É calculada usando o gráfico de fluxo de controle do programa, como M = E - N + 2P,
- Onde E é o número de arestas, N é o número de nós, e P é o número de componentes conectados.

Complexidade ciclomática

```
int CalcularDesconto(int valor) {
    if (valor > 5000) return 20;
    if (valor > 3000) return 15;
    if (valor > 1000) return 10;
    return 0;
}
```

Complexidade ciclomática

O grafo de fluxo de controle para este código seria:

- Nó 1: Início do método.
- Nó 2: Condição valor > 5000.
- Nó 3: Condição valor > 3000.
- Nó 4: Condição valor > 1000.
- Nó 5: return 0.

Complexidade ciclomática

- Nó 1: Início do método.
- Nó 2: Verificação if (valor > 5000).
- Se verdadeiro, vai para return 20.
- Se falso, vai para a próxima verificação.
- Nó 3: Verificação if (valor > 3000).
- Se verdadeiro, vai para return 15.
- Se falso, vai para a próxima verificação.

Complexidade ciclomática

- Nó 4: Verificação if (valor > 1000).
- Se verdadeiro, vai para return 10.
- Se falso, vai para return 0.
- Nó 5: Retorno final return 0.

Complexidade ciclomática

Calcule os Valores

- E (arestas): Transições entre os nós (7 no total).
- N (nós): 5 (cada bloco de código e decisão).
- P (componentes): 1 (é um único método).

Complexidade ciclomática

Aplicando a fórmula:
Complexidade Ciclomática = 7 – 5 + 2 (1) = 4

- A complexidade ciclomática é 4, o que significa que há quatro caminhos independentes que você precisaria testar para cobrir todas as condições do método. Um valor mais alto indicaria maior complexidade e mais esforço de teste.

DÚVIDAS?

Aula 05 - Métricas de Complexidade II

Disciplina: Manutenção de Software

Prof. Me. João Paulo Biazotto

UniCesumar

Profundidade de aninhamento

Número de estruturas internas, como for, while e if aninhadas

Dificulta compreensão

```
1. public int method(int n) {
2.     for(int i = 0; i < n; i++) {
3.         for(int j = 0; j < n; j++) {
4.             for(int k = 0; k < n; k++) {
5.                 ...
6.             }
7.         }
8.     }
9. }
```

Tamanho do vocabulário

Refere-se ao número de operadores e operandos distintos usados no código. Está relacionado à compreensão e complexidade do software

Mais identificadores podem significar que eles são mais significativos

Tamanho do vocabulário

```
int a = 5;
int b = a + 10;
int c = b - a;
```

Qual o tamanho do vocabulário?

Tamanho do vocabulário

```
int a = 5;
int b = a + 10;
int c = b - a;
```

Qual o tamanho do vocabulário?

- Operadores: =, +, - (3 distintos)
- Operandos: a, b, c, 5, 10 (5 distintos)

- Tamanho do vocabulário = 8

Métricas OO (CK)

Métricas de Chidamber-Kemerer (CK) específicas para sistemas orientados a objetos.

Métricas OO (CK)

Profundidade de herança (DIT)

Números de filhos (NOC)

Acoplamento entre objetos (CBO)

Falta de coesão em métodos (LCOM)

Métodos ponderados por classes (WMC)

Resposta para classe (RFC)

Profundidade da árvore de herança (DIT)

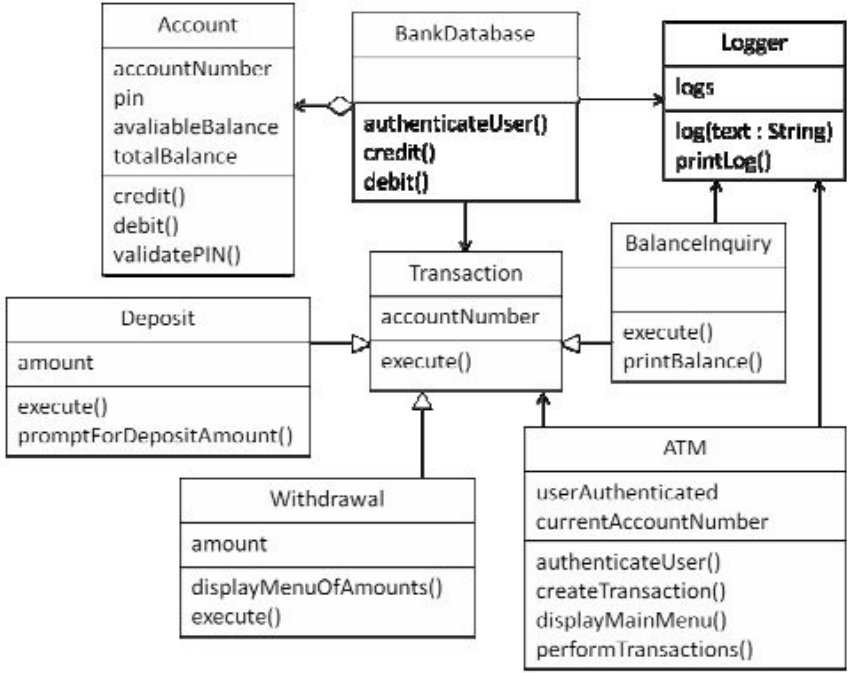
Representa o número de níveis que uma classe herda métodos e atributos

Quanto maior a profundidade

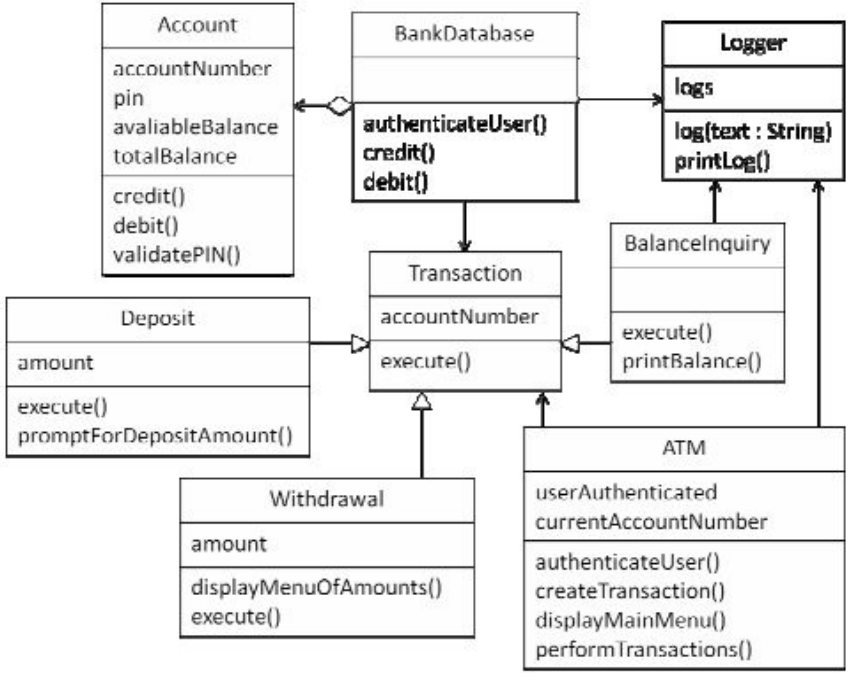
Mais complexo o projeto

Mais difícil de se entender um módulo

Profundidade da árvore de herança (DIT)



Profundidade da árvore de herança (DIT)

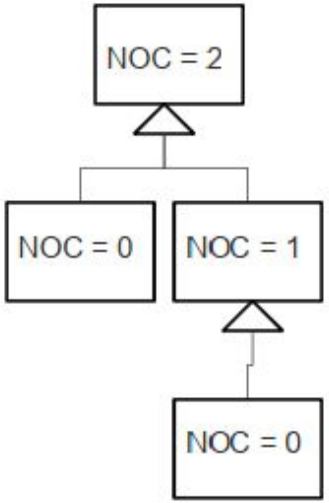


Classe	Valor
Account	1
ATM	1
BalanceInquiry	2
BankDatabase	1
Deposit	2
Logger	1
Transaction	1
Withdrawal	2

Número de filhos (NOC)

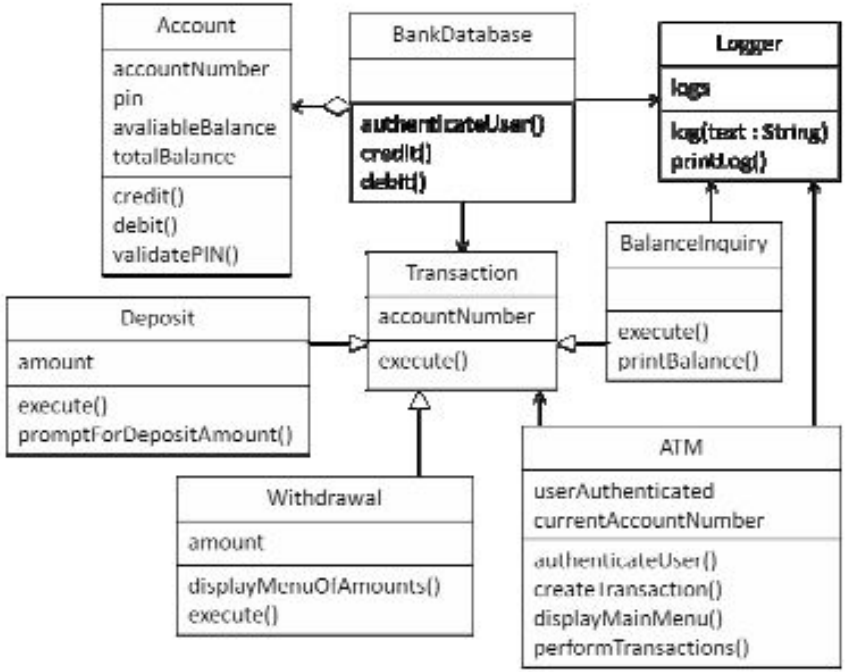
Conta o número de subclasses diretas

Mede a largura da hierarquia de uma classe

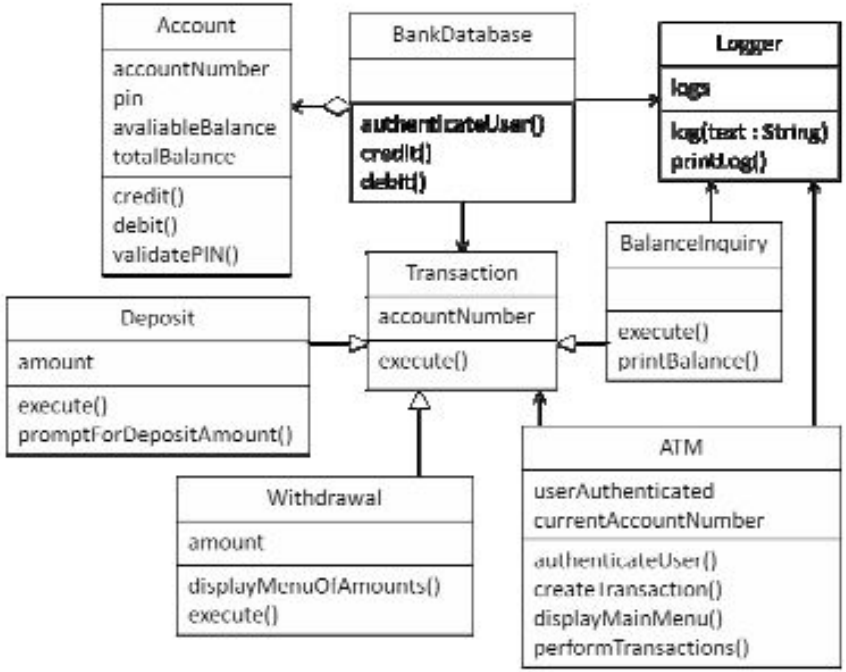


Valor alto, pode indicar reuso

Número de filhos (NOC)



Número de filhos (NOC)



Classe	Valor
Account	0
ATM	0
BalanceInquiry	0
BankDatabase	0
Deposit	0
Logger	0
Transaction	3
Withdrawal	0

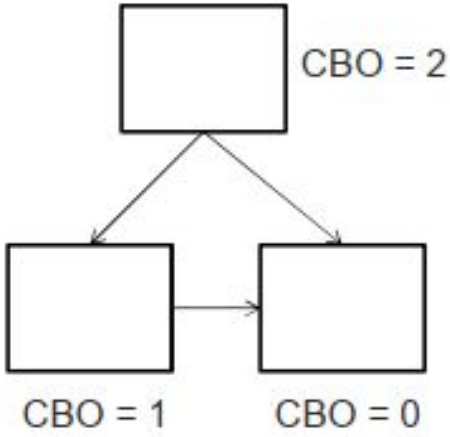
Acoplamento entre objetos (CBO)

Semelhante a Fan-out

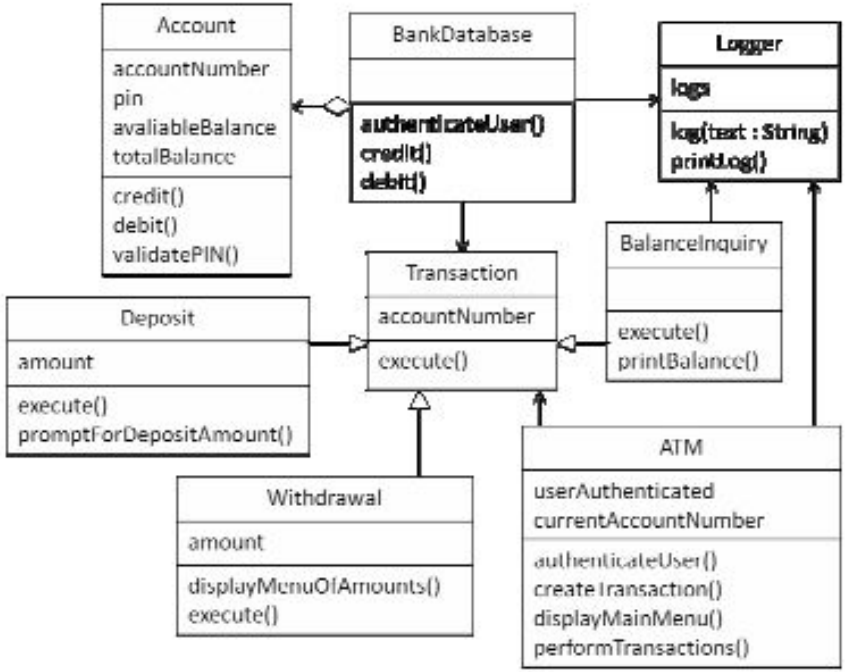
Conta classes chamadas por uma classe

Quanto mais acoplado uma classe

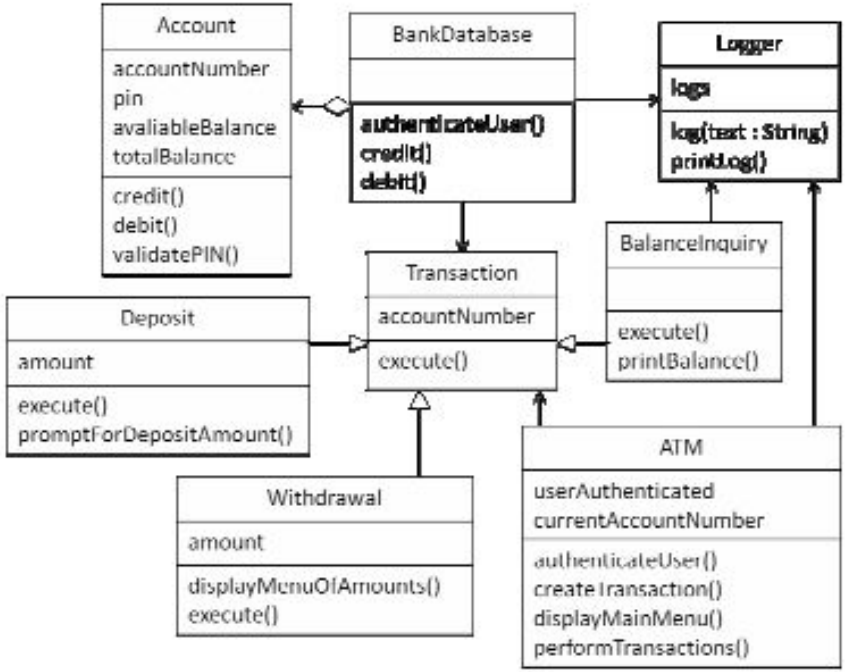
Mais difícil de entender e de manter



Acoplamento entre objetos (CBO)



Acoplamento entre objetos (CBO)



Classe	Valor
Account	0
ATM	2
BalanceInquiry	2
BankDatabase	3
Deposit	1
Logger	0
Transaction	0
Withdrawal	1

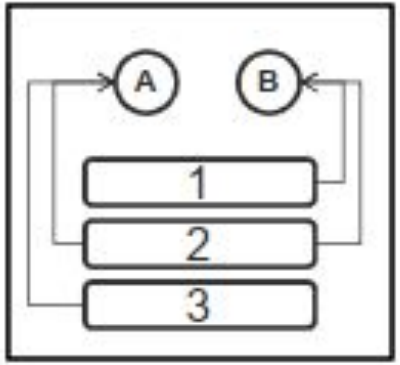
Falta de coesão em métodos (LCOM)

Para cada par de métodos, métodos que não compartilham atributos menos os que compartilham

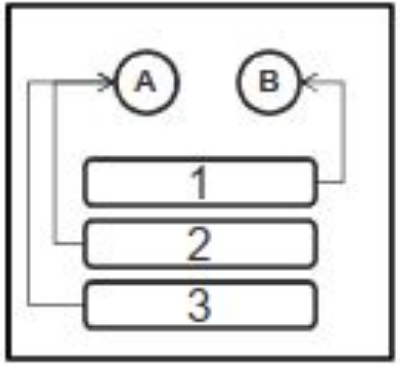
Mais atributos em comum, maior coesão, menor perda de coesão (LCOM)

Baixa coesão aumenta complexidade

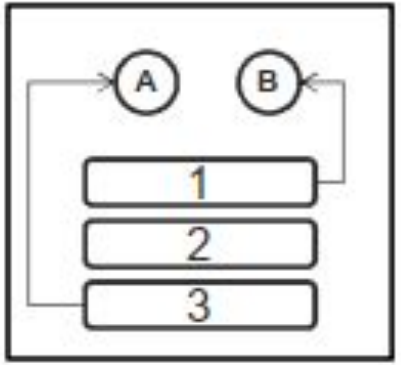
Falta de coesão em métodos (LCOM)



LCOM = 0 (1 - 2)



LCOM = 1 (2 - 1)



LCOM = 3 (3 - 0)

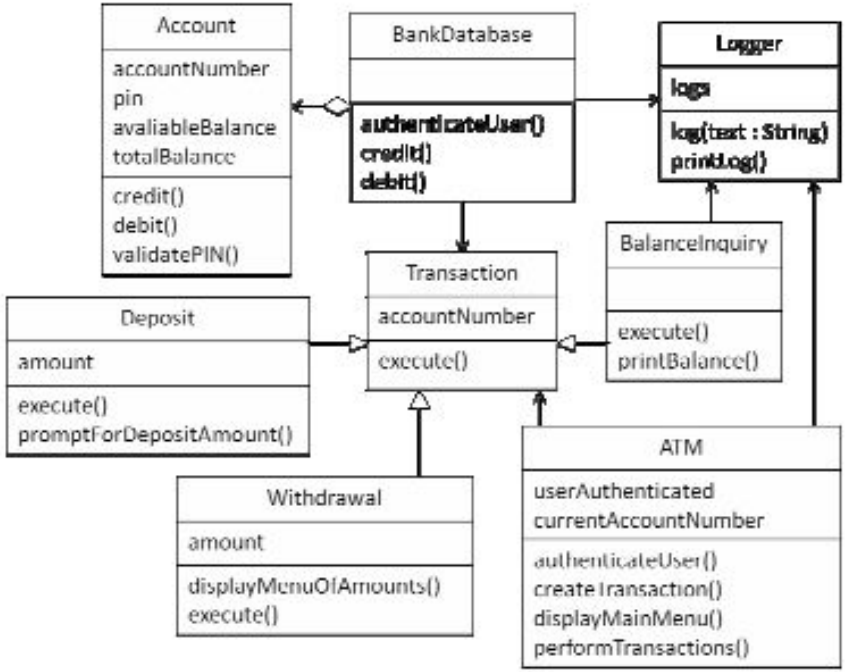
Resposta para classe (RFC)

Conta o números de métodos que podem ser executados em resposta a uma mensagem recebida por um objeto

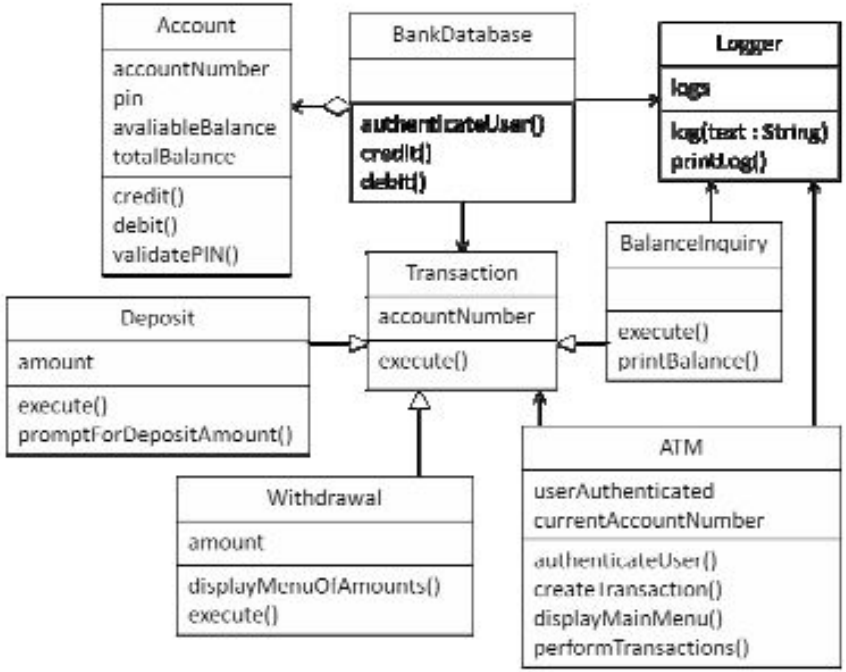
Conta o número de métodos da classe mais o número de métodos chamados pelos métodos da classe

Quanto maior o RFC, mais complexa é a classe

Acoplamento entre objetos (CBO)



Acoplamento entre objetos (CBO)



Classe	Valor
Account	3
ATM	?
BalanceInquiry	3
BankDatabase	?
Deposit	3
Logger	2
Transaction	1
Withdrawal	3

Acoplamento entre objetos (CBO)

```
public class ATM {
  ...
  private void authenticateUser() { ... }

  private void performTransactions() {
    ...
    switch ( mainMenuSelection ) {
        case DEPOSIT:
          currentTransaction.execute();
          break;
          ...
        case LOG:
          Logger.printLog();
          break;
    }
  }

  private int displayMainMenu() {
    ...
    Logger.log("User option: " + option);
    return option;
  }

  private Transaction createTransaction() { ... }
}
```

ATM
userAuthenticated currentAccountNumber
authenticateUser() createTransaction() displayMainMenu() performTransactions()

RFC = 7 (4 + 3)

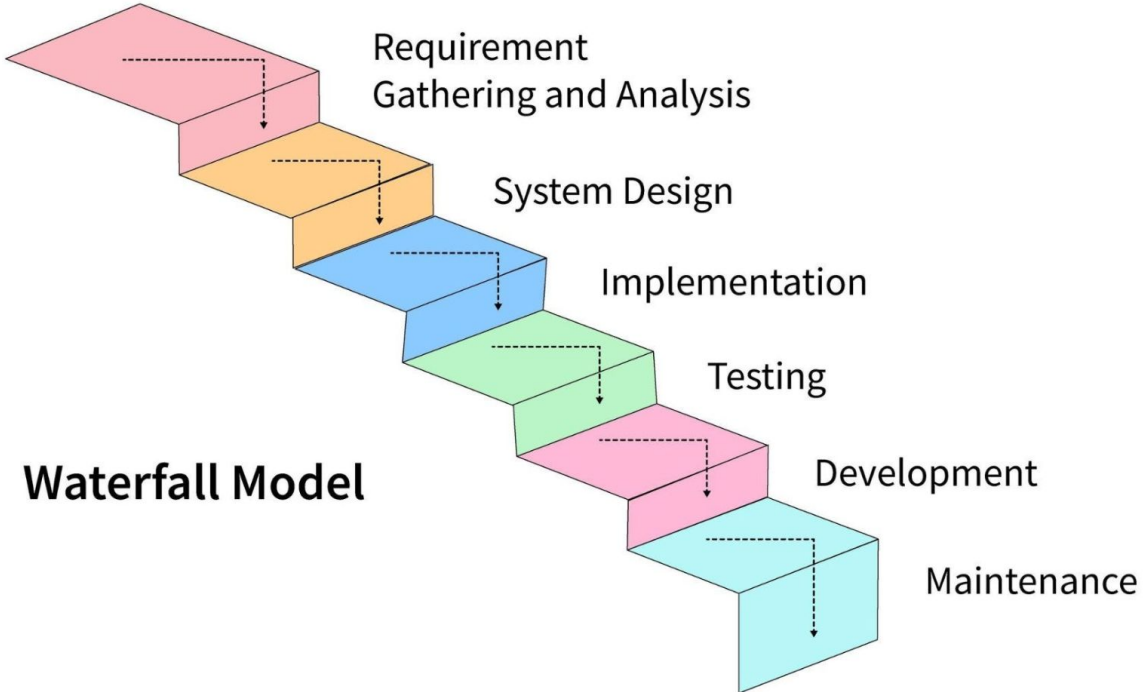
Aula 06 - Engenharia Reversa

Disciplina: Manutenção de Software

Prof. Me. João Paulo Biazotto

UniCesumar

Engenharia Progressiva



Engenharia Progressiva

(1) Software a ser produzido => *Análise* - o quê o sistema deve fazer ⇒ **Documento de Especificação**

(2) *Documento de Especificação* ⇒ *Projeto* - utiliza o documento de especificação e define como o comportamento especificado será obtido ⇒ **Documento de Arquitetura**

Engenharia Progressiva

Documento de Arquitetura ⇒ *Implementação* - Utiliza uma linguagem de programação ⇒ **Código**

Engenharia Progressiva

Processo **tradicional** de engenharia de software, caracterizado pelas atividades **progressivas do ciclo de vida**, que partem de um alto nível de abstração, para um baixo nível de abstração.

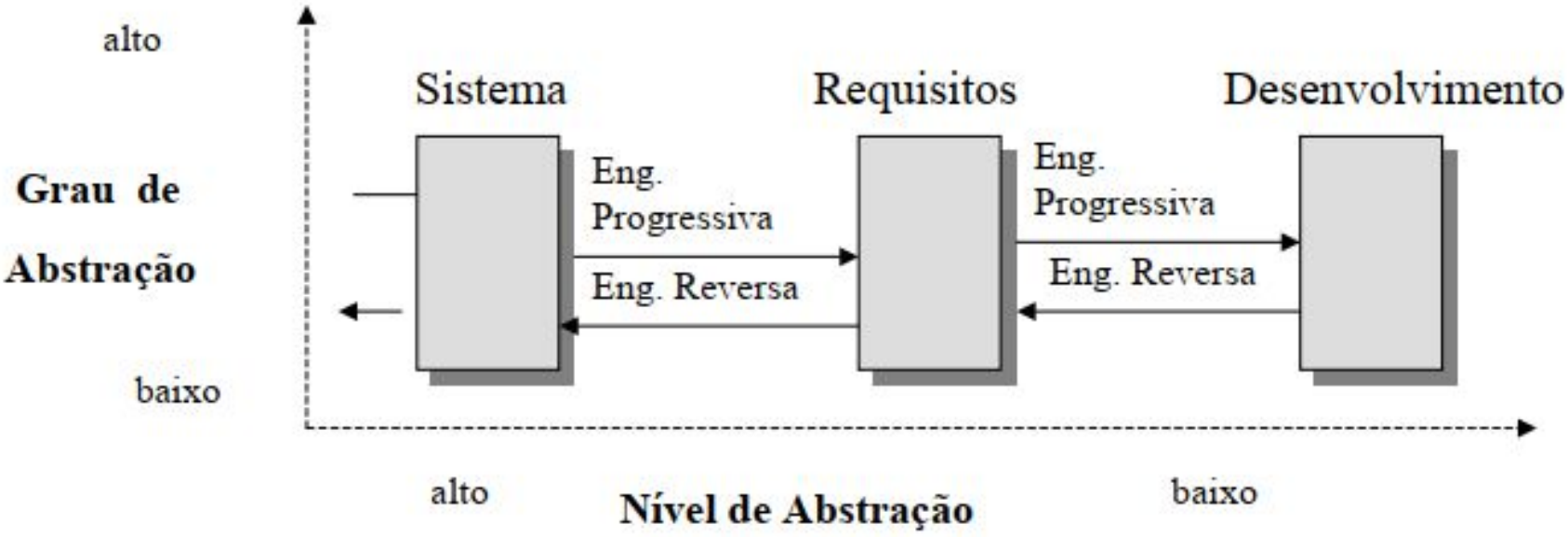
Mas é sempre assim?

- Sistemas sem documentação
- Dificuldade de manutenção
- Erros gerando outros erros
- Código duplicado

Engenharia Reversa

O processo **inverso** a Engenharia Progressiva, caracterizado pelas **atividades retroativas** do ciclo de vida, que partem de um **baixo** nível de **abstração** para um **alto** nível de **abstração**

Engenharia Progressiva x Engenharia Reversa



Engenharia Reversa

O termo “Engenharia Reversa” tem sua origem na análise de **hardware**, pois é comum a prática de decifrar projetos de produtos finalizados com intuito de **duplicá-los**.

Engenharia Reversa

O conceito de Engenharia Reversa de **Software** é similar. Porém, tradicionalmente o objetivo da dessa engenharia reversa é obter apenas um **entendimento do sistema**.

Engenharia Reversa

Processo de exame e compreensão do software existente, para recapturar ou recriar o projeto e decifrar os requisitos atualmente implementados pelo sistema, apresentando-os em um nível ou grau mais alto de abstração

Engenharia Reversa

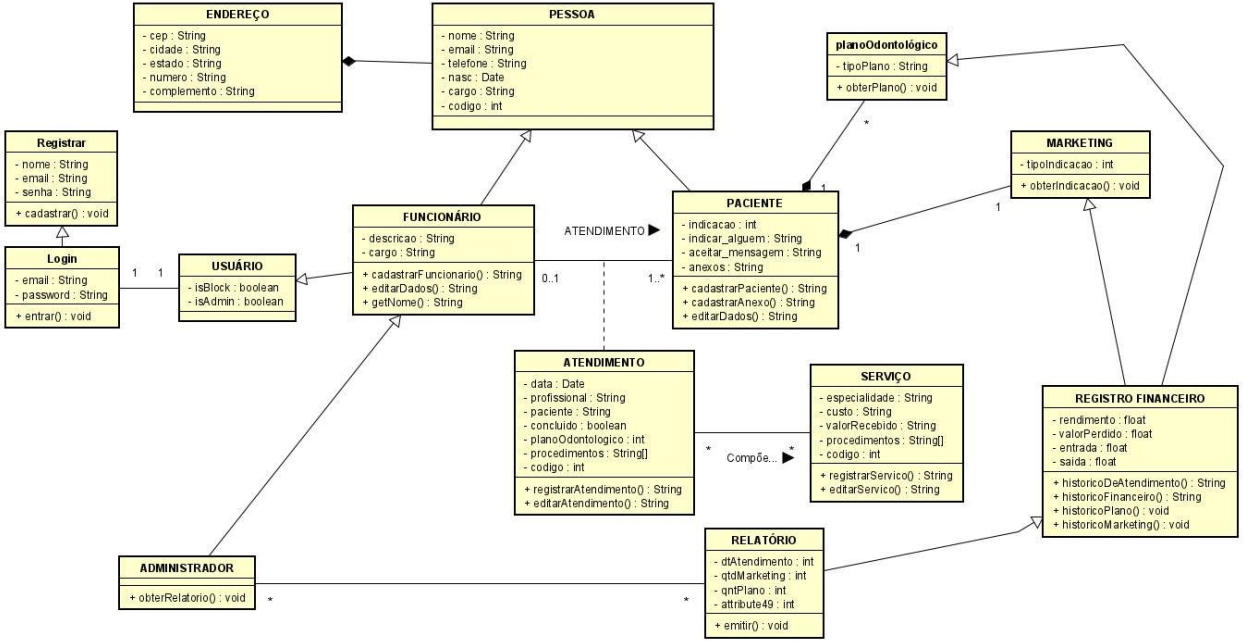
Por meio da engenharia reversa um software pode ser visualizado em diferentes níveis de abstração. Cada VISUALIZAÇÃO abstrai características próprias da fase do ciclo de vida correspondente à abstração.

Elementos da Engenharia Reversa

1) Nível de abstração

Conforme o nível de abstração aumenta, mais compreensíveis se tornam as informações

Elementos da Engenharia Reversa



Elementos da Engenharia Reversa

2) Completitude do processo

Refere-se ao nível de detalhes que é fornecido em cada nível de abstração

Elementos da Engenharia Reversa

3) Interatividade

Refere-se ao grau de participação do ser humano no processo de engenharia reversa

Conforme o nível de abstração aumenta, a interatividade deve aumentar ou a completitude será prejudicada

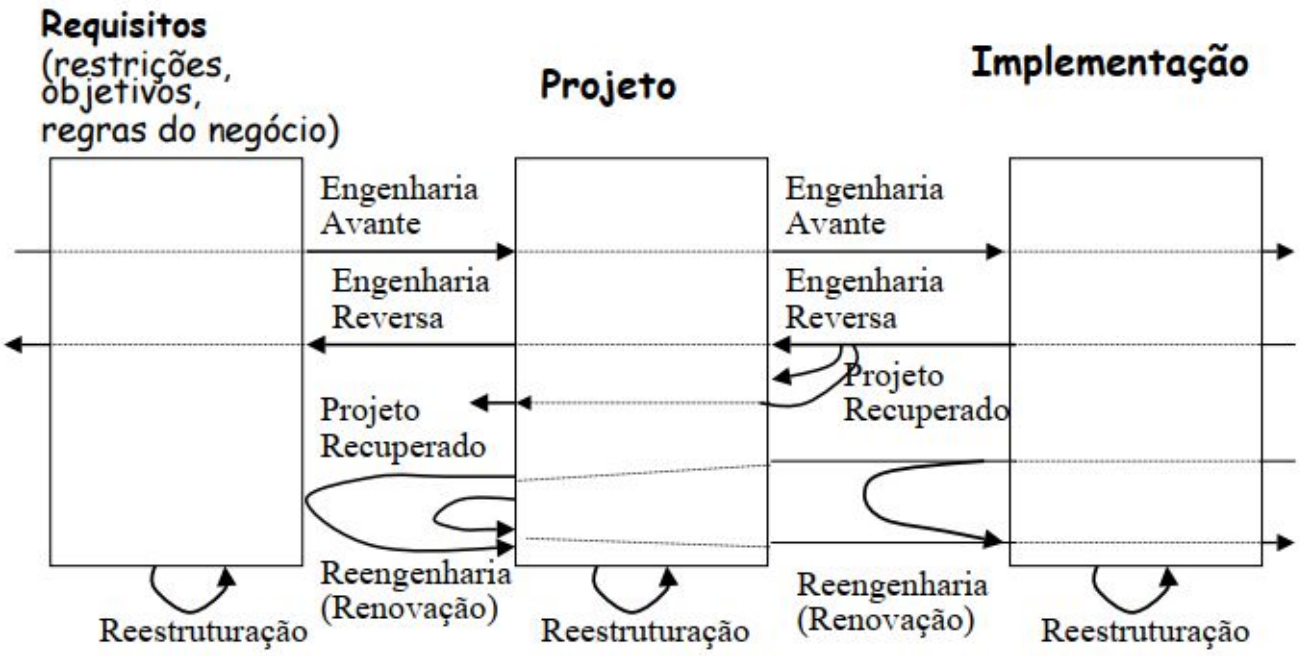
Elementos da Engenharia Reversa

4) Direcionalidade

Se a direcionalidade tem um **sentido único**, toda **informação** extraída a partir do **código** fonte é usada durante as **atividades de manutenção.**

Se a direcionalidade tem sentido **duplo**, a informação é usada para alimentar uma **ferramenta** de **reengenharia.**

Elementos da Engenharia Reversa



Entradas para Engenharia Reversa

Quais documentos utilizados para realizar a engenharia reversa?

- Código fonte
- Informações de usuário e/ou analista
- Documentação existente (manual do usuário, manual de sistema, fluxogramas, etc)

Engenharia Reversa

O que fazer com essas informações?

- Manutenção
- Mesmo paradigma e mudança de linguagem
- Mudança de paradigma (só modelo lógico)
- Mudança de paradigma e de linguagem
- Reengenharia

Engenharia Reversa

Reunir todas as informações de modo que sejam expressas através de alguma ferramenta disponível

- Pseudocódigos
- Diagrama de fluxo de dados (DFD)
- Outros métodos de análises existentes

Engenharia Reversa

Engenharia reversa x manutenção

As atividades de manutenção fornecem a motivação para muitas ferramentas de engenharia reversa. Essa motivação é proveniente da elevada proporção de tempo e custos despendida no entendimento e exame do software a ser mantido.

Engenharia reversa x manutenção

As atividades de **manutenção** fornecem a **motivação** para muitas ferramentas de engenharia reversa. Essa motivação é proveniente da elevada **proporção** de **tempo** e **custos** despendida no **entendimento** e exame do software a ser mantido.

Engenharia reversa x manutenção

Nas manutenções **adaptativas** (adequar o software a novo ambiente) e **evolutivas** (adicionar novas funcionalidades ao software), as técnicas de engenharia reversa são usadas **indiretamente**, através do fornecimento de **visões do software**, para localizar os componentes **onde** serão realizadas as **mudanças** e **adições** necessárias.

Engenharia reversa x manutenção

Para mudanças **preventivas** (redução de esforços em futuras mudanças), ferramentas de engenharia reversa podem fornecer um **discernimento** de onde e como realizar **mudanças** apropriadas, através da produção de visões do software.

Engenharia reversa x manutenção

Os maiores **benefícios** de engenharia reversa serão mais reconhecidos quando **manutenções futuras** tiverem como apoio a **documentação produzida** numa manutenção anterior.

Engenharia reversa x manutenção

Os maiores **benefícios** de engenharia reversa serão mais reconhecidos quando **manutenções futuras** tiverem como apoio a **documentação produzida** numa manutenção anterior.

Técnicas da Engenharia Reversa

- **Sem o código-fonte**
  - Análise do fluxo de dados
  - Desassemblar
  - Decompilar
- **Com o código-fonte**
  - Extrair informações
  - Código
  - Dados
  - Documentação
  - Outras fontes

Técnicas da Engenharia Reversa

- **Sem o código-fonte**
  - Análise do fluxo de dados
  - Desassemblar
  - Decompilar
- **Com o código-fonte**
  - Extrair informações
  - Código
  - Dados
  - Documentação
  - Outras fontes

Análise de fluxo de dados

Analisar o comportamento dos dados trafegados na na rede pode ajudar a produzir uma nova implementação do software que imita o mesmo comportamento. Isto é especialmente utilizado na engenharia reversa de drivers de dispositivos.

Desassemblar

Usando um **desassembler**, conseguimos obter a linguagem de máquina diretamente do programa. Isto funciona em qualquer programa de computador, mas pode levar um bom tempo, especialmente para alguém que não esteja acostumado ao código de máquina.

Decompilação

Neste método utiliza-se um **decompilador**, um programa que **tenta recriar o código-fonte** em uma linguagem de alto nível, tendo disponível apenas o código de máquina.

Tipos de Engenharia Reversa

- **Com o código-fonte**
  - Extrair informações
  - Código
  - Dados
  - Documentação
  - Outras fontes

Extrair Informações

O primeiro trabalho que se deve fazer é **coletar informações** sobre o sistema a ser estudado.

As informações podem ser extraídas de várias fontes: o **código fonte**, a **execução**, os **dados** (por exemplo, em banco de dados), a “documentação”, ou outras fontes.

Análise Estática

A primeira fonte, o código, é a mais usada. A análise do código é chamada também de análise estática (por oposição à análise dinâmica que é a execução do sistema).

Análise Estática

Quais são os **componentes** básicos do sistema: **arquivos**, rotinas, tipos, **variáveis**, classes, etc;

Relações de definição conectam um componente com seu conteúdo (onde ele se encontra);

Relações de **referência** conectam um componente com aqueles que o usam (ex. se uma rotina A chamar uma outra rotina B, então **A depende de B**)

Análise Estática

Ferramentas para fazer essa análise são chamadas de “parser” (análise sintática). O “parsing” é a primeira etapa da compilação do código fonte. Para fazer isso, é preciso conhecer a sintaxe da linguagem de programação usada. Dependendo das necessidades, pode-se usar parsers específicos que vão procurar só um tipo particular de informação.